

AgentShield Backend Walkthrough

Generated from code inspection of the current project in 'app/'.

1. Project Overview

AgentShield is a FastAPI backend that analyzes software dependencies from three main target types:

- single package or multi-package input
- dependency file input
- GitHub repository input

Its core job is to turn those inputs into a normalized dependency inventory, run vulnerability lookups against exact-version packages, optionally enrich the result with metadata such as licenses and repository facts, and then return both:

- a full backend JSON report
- a compact reasoning-oriented 'llm_context' used by '/agent/chat'

The project currently supports these major capabilities:

- single package scan
- multi-package scan
- dependency file scan
- pasted dependency-list text auto-treated as file-like input
- GitHub repo scan
- GitHub repo scan with token support for private repositories
- OSV-based vulnerability lookups
- NVD CVE enrichment
- deps.dev package metadata enrichment
- GitHub SBOM acquisition with parser fallback
- LLM-assisted '/agent/chat' request interpretation and answer generation

At a high level, the project mixes two eras of backend design:

- the current main path: 'run_unified_scan()' in 'app/agents/unified_scan_agent.py'
- older compatibility paths: '/scan/github' and '/scan/web'

The current production-style architecture is centered on the unified scan graph and the '/agent/chat' controller.

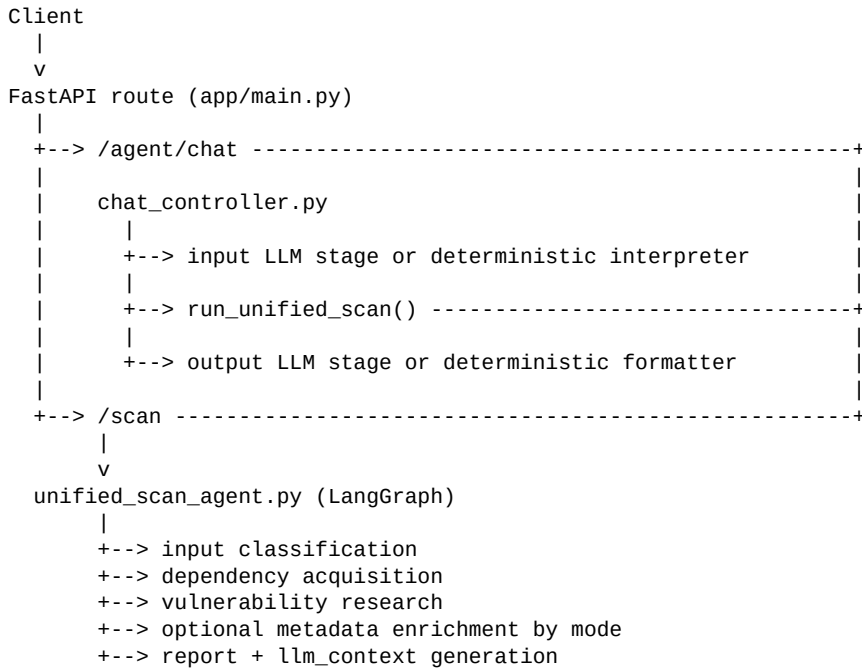
2. High-Level Architecture

The codebase is organized into these major layers:

- FastAPI routes in 'app/main.py'
- controller layer in 'app/controller/' for '/agent/chat'
- deterministic scan orchestration in 'app/agents/unified_scan_agent.py'
- service layer in 'app/services/'
- parsing utilities in 'app/parsers/' and 'app/utils/repo_utils.py'
- schemas and request/response models in 'app/schemas/' and 'app/models/'

- reporting layer in 'app/services/reporting.py'

High-level architecture flow:



Backend execution layers in order:

1. request model validation by FastAPI/Pydantic
2. route handler
3. controller interpretation for '/agent/chat' or direct backend normalization for '/scan'
4. LangGraph state machine
5. provider/service calls
6. report construction
7. optional LLM output formatting

3. Endpoints

'/agent/chat'

- File: 'app/main.py'
- Request model: 'AgentChatRequest' in 'app/models/models.py'
- Purpose: conversational endpoint that accepts natural language plus optional structured fields
- Input types:
- package / packages
- repo URL
- 'file_name' + 'file_content'
- token for private repo
- Internal trigger:
- 'handle_chat_request()' in 'app/controller/chat_controller.py'

- Returns:
- conversational answer
- whether backend was called
- normalized backend payload
- mode / mode source
- input type
- 'response_meta'
- optional 'scan_result' when debug is requested

Important note:

'/agent/chat' does not make an internal HTTP call to '/scan'. It directly calls 'run_unified_scan()' through Python function invocation.

'/scan'

- File: 'app/main.py'
- Request model: 'UnifiedScanRequest'
- Purpose: deterministic backend scan API
- Input types:
- repo URL
- package(s)
- dependency file represented as 'file_name' + 'file_content'
- Internal trigger:
- 'run_unified_scan()' in 'app/agents/unified_scan_agent.py'
- Returns:
- 'json_report'
- 'summary'

This is the most important backend endpoint for current architecture.

'/scan/github'

- File: 'app/main.py'
- Request model: 'GithubScanRequest'
- Purpose: older GitHub-only compatibility scan path
- Internal trigger:
- 'run_github_scan()' in 'app/services/github_scanner.py'
- Returns:
- dependency analysis report for repo acquisition only

This is a legacy-style path compared with the unified scan graph.

'/scan/web'

- File: 'app/main.py'
- Request model: 'WebScrapeRequest'
- Purpose: older follow-on enrichment path using a separate LangGraph workflow
- Internal trigger:
- 'run_web_scrape()' in 'app/agents/web_scraping_agent.py'
- Returns:
- report containing package maps, vulnerability research, and OCI analysis fields

This endpoint is still present, but it is architecturally separate from the current '/scan' path.

Which endpoint handles what

- package scan: '/scan' or '/agent/chat'
- repo URL scan: '/scan' or '/agent/chat'
- file upload equivalent: '/scan' or '/agent/chat' using 'file_name' + 'file_content'
- legacy GitHub-only repo scan: '/scan/github'
- legacy web follow-up analysis: '/scan/web'

4. Input Types and Flow

Single package

Entry:

- '/agent/chat': natural language or structured fields
- '/scan': direct structured payload

Flow:

```
request
-> controller interpretation or request normalization
-> InputType.PACKAGE
-> acquire_direct_package_inventory()
-> enrich_vulnerabilities()
-> optional deps.dev / GitHub metadata by mode
-> report + llm_context
-> formatter / output LLM
```

Multiple packages

Entry:

- '/agent/chat' with message like multiple package request
- '/scan' with 'packages: [...]'

Flow is the same as single package, but 'acquire_direct_package_inventory()' creates multiple normalized dependencies.

Dependency file

Entry:

- '/scan' with 'file_name' and 'file_content'
- '/agent/chat' with same structured fields
- natural language text can also become file-like input if it looks like a dependency list

Flow:

```
request
-> detect dependency file input
-> acquire_file_dependency_inventory()
-> parse_dependency_file()
-> normalized dependency inventory
-> classify discovery quality
-> vulnerability research if exact versions exist
-> optional enrichments by mode
```

Pasted dependency-list text

This is handled in two places:

- 'app/controller/interpreter.py' for '/agent/chat'
- 'app/services/input.py' for '/scan'

If raw text looks like multiple dependency lines, the system infers:

- 'file_name = "requirements.txt"'
- 'file_content = raw_text'

So the pasted text is treated like a synthetic dependency file.

GitHub repo URL

Entry:

- '/scan' with 'repo_url'
- '/agent/chat' with repo wording or structured repo URL

Flow:

```
request
-> InputType.GITHUB_REPO
-> acquire_github_dependency_inventory()
    -> try GitHub SBOM
    -> fallback to repo file parsing
-> vulnerability research
-> optional enrichments by mode
-> report + llm_context
```

GitHub repo URL + token

Same as repo flow, but the token affects:

- GitHub API repo access
- SBOM fetch authorization
- file content fetch strategy for private repos

Private repo content fetch uses GitHub contents API in 'fetch_file_content()' rather than raw GitHub URLs.

5. '/agent/chat' Flow in Code

Primary files:

- 'app/main.py'
- 'app/controller/chat_controller.py'
- 'app/controller/llm_input.py'
- 'app/controller/llm_output.py'
- 'app/controller/interpreter.py'
- 'app/controller/llm.py'
- 'app/controller/prompts.py'

Request entry

'app/main.py' defines:

- '@app.post("/agent/chat")'

The route builds the argument set and calls:

- 'handle_chat_request(...)'

Prompt loading

'app/controller/prompts.py' handles prompt loading.

- 'load_agent_prompt()'
- 'load_input_prompt()'
- 'load_output_prompt()'

Environment overrides:

- 'AGENT_CONTROL_PROMPT_PATH'
- 'AGENT_INPUT_PROMPT_PATH'
- 'AGENT_OUTPUT_PROMPT_PATH'

If a prompt file cannot be read, the loader returns an empty string.

LLM availability

'chat_controller.py' calls:

- 'get_default_llm_with_status()' from 'app/controller/llm.py'

That function:

1. gathers diagnostics
2. tries to create 'OciAgentLLM'
3. returns '(llm, diagnostics)' or '(None, diagnostics)'

If 'REQUIRE_AGENT_LLM=true', startup failure becomes hard failure instead of silent fallback.

Input LLM stage

'chat_controller.py' calls:

- 'run_input_llm_stage(...)'

Behavior:

- if LLM exists: load input prompt and ask the model for a JSON decision
- if no LLM: use deterministic interpretation via 'deterministic_input_decision()'

Decision contract:

- 'action = "clarify"' with 'question'
- 'action = "execute"' with 'backend_payload'

Clarification vs execute

In 'llm_input.py':

- 'validate_input_decision()' validates the LLM output
- if invalid, a 'ValueError' is raised

In deterministic fallback:

- 'interpret_request()' in 'interpreter.py' infers target, mode, and missing fields
- if information is missing, it sets 'clarification_question'

Typical clarification cases:

- package name present but exact version missing
- repo scan requested for private repo without token
- dependency file name or contents missing
- more than one target appears in one request

Backend payload validation

'validate_input_decision()' enforces:

- exactly one target
- valid mode among 'fast', 'detailed', 'developer'
- complete package entries
- file payload must include 'file_content'

Backend execution

If action is 'execute', 'chat_controller.py' directly calls:

- 'scan_executor(**payload)'

Default executor:

- 'run_unified_scan'

This means '/agent/chat' and '/scan' share the same backend scan engine.

Output LLM stage

After backend execution:

- 'run_output_llm_stage(...)' is called

Behavior:

- if LLM exists: use 'load_output_prompt()' and pass 'llm_context'
- if no LLM: use deterministic formatter 'format_chat_response()'

Final response assembly

'chat_controller.py' adds:

- 'status'
- 'message'
- 'backend_called'
- sanitized 'backend_payload'
- 'mode'
- 'mode_source'
- 'input_type'
- 'controller_prompt_loaded'
- 'response_meta'

Optional:

- 'scan_result' when 'include_debug=True'

Sensitive fields such as tokens are redacted by '_sanitize_for_response()'.

'/agent/chat' sequence diagram

```
Client
-> /agent/chat
-> AgentChatRequest parsed by FastAPI
-> handle_chat_request()
  -> load prompt status
  -> get_default_llm_with_status()
  -> run_input_llm_stage()
    -> LLM path OR deterministic interpreter
  -> if clarify:
    return needs_clarification response
  -> else execute:
    run_unified_scan()
    run_output_llm_stage()
      -> LLM path OR deterministic formatter
    build response_meta
    return final chat response
```

Fallback behavior when LLM is unavailable

If OCI LLM cannot be created:

- input stage falls back to deterministic request interpretation
- output stage falls back to deterministic response formatter
- response metadata includes:
 - 'llm_input_stage'
 - 'llm_output_stage'
 - 'llm_available'
 - 'llm_fallback_reason'
 - 'llm_diagnostics'

This is why '/agent/chat' still works even without live LLM access.

6. '/scan' Flow in Code

Primary files:

- 'app/main.py'
- 'app/agents/unified_scan_agent.py'
- 'app/services/input.py'
- 'app/services/dependency_inputs.py'
- 'app/services/github/acquisition.py'
- 'app/services/vulnerability/research.py'
- 'app/services/enrichment/'
- 'app/services/reporting.py'

Request parsing

'/scan' receives 'UnifiedScanRequest' and calls:

- 'run_unified_scan(...)'

Request normalization

In 'run_unified_scan()':

- 'normalize_request_context(...)' converts raw inputs into 'ScanRequestContext'
- 'detect_input_type()' assigns:
 - 'GITHUB_REPO'
 - 'PACKAGE'
 - 'DEPENDENCY_FILE'
 - 'UNKNOWN'

Graph execution

'run_unified_scan()' invokes the compiled LangGraph workflow:

- 'unified_scan_agent.invoke(initial_state)'

Dependency acquisition split

Input-specific branches:

- repo -> 'acquire_github_dependency_inventory()'
- package -> 'acquire_direct_package_inventory()'
- file -> 'acquire_file_dependency_inventory()'

Vulnerability research

'query_vulnerabilities_node()' calls:

- 'enrich_vulnerabilities(...)'

This uses:

- OSV for exact-version package checks
- NVD for CVE enrichment on discovered CVEs

Metadata enrichment by mode

After vulnerabilities:

- 'fast': stop and format
- 'detailed': add deps.dev
- 'developer': add deps.dev, then GitHub repo metadata
- 'full': same branch as developer inside the graph

Report generation

'format_output_node()' builds:

- backend 'report'
- compact 'llm_context'
- text 'summary'

The backend report contains full raw structures for API/debug use.

The 'llm_context' is the smaller reasoning-oriented representation used by the agent output stage.

'/scan' flowchart

```

/scan
-> run_unified_scan()
-> normalize_request_context()
-> LangGraph:
    classify_request
    -> acquire_repo_dependencies
    -> acquire_package_dependency
    -> acquire_file_dependencies
    route_after_inventory
    -> discovery_only ? format_output
    -> else query_vulnerabilities
    route_by_mode_after_vulns
    -> fast -> format_output
    -> detailed/developer/full -> enrich_deps_dev
    route_after_deps_dev
    -> detailed -> format_output
    -> developer/full -> enrich_github_api

```

```
format_output  
-> UnifiedScanResponse
```

7. LangGraph Nodes and Mode Flow

Primary graph file:

- 'app/agents/unified_scan_agent.py'

State shape:

- 'UnifiedScanState'
- 'context'
- 'route'
- 'inventory'
- 'vulnerability_research'
- 'metadata_enrichment'
- 'report'
- 'summary'

Nodes

'classify_request'

- File: 'app/agents/unified_scan_agent.py'
- Function: 'classify_request_node'
- Role:
 - infer or preserve scan mode
 - write 'route' metadata into state

'acquire_repo_dependencies'

- Function: 'acquire_repo_dependencies_node'
- Calls:
 - 'acquire_github_dependency_inventory()'
- Role:
 - build dependency inventory from repo

'acquire_package_dependency'

- Function: 'acquire_package_dependency_node'
- Calls:
 - 'acquire_direct_package_inventory()'
- Role:
 - build dependency inventory from package input

'acquire_file_dependencies'

- Function: 'acquire_file_dependencies_node'
- Calls:
 - 'acquire_file_dependency_inventory()'
- Role:

- build dependency inventory from file content

'query_vulnerabilities'

- Function: 'query_vulnerabilities_node'
- Calls:
 - 'enrich_vulnerabilities()'
- Role:
 - run OSV and NVD research

'enrich_deps_dev'

- Function: 'enrich_deps_dev_node'
- Calls:
 - 'enrich_with_deps_dev(inventory, max_packages=None)'
- Role:
 - license / package metadata enrichment for exact-version packages

'enrich_github_api'

- Function: 'enrich_github_api_node'
- Calls:
 - 'enrich_github_repo()'
- Role:
 - repo-level GitHub metadata enrichment

'format_output'

- Function: 'format_output_node'
- Role:
 - build final JSON report and summary
 - build 'llm_context'

Routing functions

- 'route_by_input()'
- 'route_after_inventory()'
- 'route_by_mode_after_vulns()'
- 'route_after_deps_dev()'

Modes are implemented as conditional graph routing, not just formatting flags.

Mode behavior summary

Fast mode

- acquisition
- vulnerability research
- reporting
- no deps.dev
- no GitHub metadata enrichment

Detailed mode

- Fast mode behavior
- plus deps.dev enrichment
- richer package-level 'package_analysis'

Developer mode

- Detailed mode behavior
- plus GitHub repo metadata enrichment
- plus 'provider_plan' in report

Full mode

- schema and graph support exist
- same enrichment branch as Developer mode
- not naturally produced by '/agent/chat' deterministic/LLM input stage, which only validates 'fast', 'detailed', 'developer'

LangGraph node flow diagram



8. Provider and Service Integration

GitHub SBOM

- Files:
- 'app/services/github/client.py'
- 'app/services/github/acquisition.py'
- Called from:
- 'acquire_github_dependency_inventory()'
- Role:
- first-choice repo dependency acquisition path
- Returns:
- 'DependencyInventory' populated from SBOM package/components

Parser fallback for repo scans

- Files:
- 'app/services/github/acquisition.py'
- 'app/utils/repo_utils.py'
- 'app/parsers/dependency_parsers.py'
- Called when:
- SBOM fetch fails or is unparseable
- Role:
- detect dependency files in repo tree
- fetch their contents
- parse them into dependencies

OSV

- File:
- 'app/services/vulnerability/osv.py'
- Called from:
- 'enrich_vulnerabilities()' in 'research.py'
- Role:
- exact-version vulnerability lookup by package + ecosystem + version
- Returns:
- advisory IDs
- CVE aliases
- severity
- advisory summaries

NVD

- File:
- 'app/services/vulnerability/nvd.py'
- Called from:
- 'enrich_vulnerabilities()' after OSV
- Role:
- CVE detail enrichment for discovered CVEs
- Returns:
- published/modified timestamps
- severity
- base score

- English description

deps.dev

- File:
- 'app/services/enrichment/deps_dev.py'
- Called from:
- 'enrich_deps_dev_node()'
- Role:
- exact-version package metadata enrichment
- Returns:
- licenses
- advisory keys
- links
- publishedAt
- isDefault

GitHub API repo metadata

- Files:
- 'app/services/enrichment/github_repo.py'
- 'app/services/github/client.py'
- Called from:
- 'enrich_github_api_node()'
- Role:
- repo-level metadata such as stars, forks, default branch, timestamps

OCI LLM

- File:
- 'app/controller/llm.py'
- Called by:
- '/agent/chat' input stage
- '/agent/chat' output stage
- Role:
- agent interpretation and answer generation

OCI analysis in legacy '/scan/web'

- Files:
- 'app/agents/web_scraping_agent.py'
- 'app/services/enrichment/oci.py'
- Role:
- older enrichment branch for the legacy web scan path

9. Important Files and Responsibilities

'app/main.py'

- FastAPI app creation
- CORS setup
- request logging middleware

- endpoint registration
- startup LLM diagnostics print

'app/controller/chat_controller.py'

- main orchestration for '/agent/chat'
- chooses LLM vs deterministic fallback
- sanitizes secrets
- builds final response payload

'app/controller/interpreter.py'

- deterministic natural-language interpretation
- target inference
- mode inference
- clarification generation

'app/controller/llm_input.py'

- validates structured LLM decision
- bridges between input LLM and deterministic fallback

'app/controller/llm_output.py'

- builds output LLM payload
- falls back to deterministic formatter if LLM is absent

'app/controller/formatter.py'

- deterministic response writer for Fast, Detailed, Developer
- consumes 'llm_context'

'app/controller/llm.py'

- OCI LLM client creation
- diagnostics
- strict-mode behavior

'app/agents/unified_scan_agent.py'

- main LangGraph workflow for modern scanning
- most important backend orchestration file

'app/services/input.py'

- request normalization for '/scan'
- package/file/repo input classification

'app/services/dependency_inputs.py'

- package input inventory builder
- file input inventory builder

‘app/services/github/acquisition.py’

- repo acquisition engine
- GitHub SBOM path
- parser fallback path

‘app/services/github/client.py’

- thin client around GitHub repo metadata, SBOM, structure, file fetch

‘app/services/vulnerability/research.py’

- exact-version filtering
- OSV parallel queries
- NVD enrichment
- skipped-package tracking

‘app/services/reporting.py’

- full report construction helpers
- compact ‘llm_context’
- package analysis for Detailed mode

‘app/services/enrichment/deps_dev.py’

- exact-version package metadata enrichment

‘app/parsers/dependency_parsers.py’

- filename-based parser dispatch

‘app/utils/repo_utils.py’

- repo structure traversal
- dependency file discovery
- actual low-level parsing helpers for many file types

‘app/schemas/scan.py’

- shared backend data contracts
- enums
- inventory, vulnerability, enrichment, and request-context models

‘app/models/models.py’

- FastAPI request/response models
- also contains older compatibility models

10. How File Upload Works

Current backend behavior is not multipart binary upload based.

The backend expects file input as:

- 'file_name'
- 'file_content'

That is true for both:

- '/scan'
- '/agent/chat'

So if a UI allows file upload, the UI must extract or read the uploaded file and send:

- the filename
- the file text contents

Flow:

```
UI uploads file
-> UI reads file contents
-> sends file_name + file_content to backend
-> backend detects dependency_file input
-> acquire_file_dependency_inventory()
-> parse_dependency_file()
-> inventory -> vulnerabilities -> enrichments -> report
-> response returned directly or via /agent/chat formatter
```

Important:

The backend currently does not expose multipart/form-data file upload handling in 'app/main.py'.

11. Current Limitations and Important Notes

Exact-version dependency requirement

Vulnerability enrichment is intentionally strongest for exact versions.

If versions are:

- missing
- ranged
- indirect and unresolved

then:

- OSV matching is skipped
- deps.dev exact-version enrichment is skipped
- result confidence drops

Repo fallback limitations

GitHub repo scanning prefers SBOM.

If SBOM is unavailable:

- the system falls back to file detection and parsing
- success depends on whether supported dependency files are found and parseable

Parser support limits

Parser dispatch is currently filename-based.

Supported top-level filenames include:

- 'requirements.txt'
- 'requirements-dev.txt'
- 'requirements-prod.txt'
- 'poetry.lock'
- 'Pipfile.lock'
- 'package-lock.json'
- 'package.json'
- 'pyproject.toml'
- Conda YAML files
- 'pom.xml'

If dependency data lives elsewhere or uses unsupported formats, it will not be parsed.

'/agent/chat' mode limitations

The '/agent/chat' input validation only accepts:

- 'fast'
- 'detailed'
- 'developer'

The schema has additional modes such as 'full', 'github_dependencies', and 'web_enrichment', but chat interpretation does not route users into those modes naturally.

README mismatch

The root README is primarily frontend- and dashboard-oriented.

It does not describe the current FastAPI backend, LangGraph scan engine, or '/agent/chat' controller flow.

Legacy endpoint split

The current main architecture is '/scan' plus '/agent/chat'.

'/scan/github' and '/scan/web' still exist, but they reflect older patterns and separate workflows.

LLM dependency

The agent can run without OCI LLM because it falls back deterministically, but that means:

- request interpretation becomes rule-based
- output wording becomes template/formatter based

12. How to Explain This in a Demo

Use this as the short speaking version:

1. The system accepts three main scan targets: package, dependency file, or GitHub repo.
2. Everything is normalized into a shared dependency inventory model.
3. The main scan engine is a LangGraph workflow that branches by input type.
4. For repos, it first tries GitHub SBOM, then falls back to parsing dependency files from the repo if needed.
5. Vulnerability matching is done mainly with OSV, and CVE details are enriched from NVD.
6. In Detailed and Developer modes, the backend also enriches exact-version packages with deps.dev metadata like licenses and useful links.
7. '/agent/chat' sits on top of the same backend and can use LLM stages for interpretation and answer writing, but it can also fall back to deterministic behavior if the LLM is unavailable.
8. The backend returns both a full JSON report and a compact 'llm_context', so the agent can respond cleanly without dumping raw provider payloads.

Demo script by input type

Package query

â When a user asks about a package, the controller or scan endpoint converts that into an exact package coordinate, runs vulnerability research, enriches metadata by mode, and then returns either a direct scan result or a conversational answer.â

Repo URL

â When a repo URL is sent, the system first tries GitHub SBOM. If that fails, it inspects the repo tree, fetches supported dependency files, parses them, and continues the scan from that normalized dependency list.â

Dependency file upload

â The backend expects the filename and file contents, not a raw binary upload. Once it receives those, it routes the content through the parser layer and then through the same vulnerability and enrichment pipeline.â

LLM interaction

â The LLM does not replace the backend scan logic. It only helps interpret user intent on the way in and generate a cleaner explanation on the way out. The actual vulnerability and metadata work stays deterministic in the backend.â

13. Final Notes

Most important current code path:

- 'app/main.py' -> '/scan'
- 'app/agents/unified_scan_agent.py'

Most important conversational path:

- 'app/main.py' -> '/agent/chat'
- 'app/controller/chat_controller.py'
- 'app/agents/unified_scan_agent.py'

If you are explaining the system to a technical audience, focus on:

- shared inventory normalization
- graph routing by input type and mode
- exact-version requirement for high-confidence enrichment
- 'llm_context' as the bridge between deterministic backend logic and user-facing agent output